

**Výstupy ŠVP****INF-08b** V programu používám opakování (cykly) a seznamy. (Informatika 4, 2/2 · z INF-08)**INF-07a** Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešení problém ten nejvhodnější. (Informatika 4, 2/2 · z INF-07)**Učivo:** testování, optimalizace – vliv vstupních dat na spotřebované výpočetní zdroje**Základní pojmy****Simulace** – Napodobení náhodného děje programem; pokus opakujeme mnohokrát a výsledky vyhodnotíme statisticky.**Pseudonáhodná čísla** – Počítač je vyrábí deterministickým algoritmem ze „semínka“; `random.seed(7)` zajistí stejnou posloupnost při každém spuštění.**Funkce modulu random** – `randint(a, b)` celé číslo  $a$  až  $b$  včetně; `randrange(a, b)` bez  $b$ ; `random()` desetinné z  $[0; 1)$ ; `choice(s)` náhodný prvek.**Relativní četnost** – Podíl počtu výskytů jevu  $A$  a počtu všech pokusů; odhad pravděpodobnosti.**Klasická pravděpodobnost** –  $P(A) = \frac{m}{n}$  – počet příznivých ku počtu všech stejně možných výsledků (H53).**Zákon velkých čísel** – S rostoucím počtem pokusů se relativní četnost blíží pravděpodobnosti – ale nikdy „přesně“.**Doba výpočtu** – Měří se `time.perf_counter()` před a po výpočtu; u simulace roste přímo úměrně počtu opakování.**Paměťová náročnost** – Seznam všech  $N$  hodů zabírá paměť úměrnou  $N$ ; šest počítadel stačí pro četnosti při libovolném  $N$ .**Metoda Monte Carlo** – Odhad veličiny (např.  $\pi$ ) pomocí velkého počtu náhodných pokusů.**Vzorce a vztahy** $f = \frac{n_A}{N}$  relativní četnost jevu  $A$  po  $N$  pokusech

$$\sigma = \sqrt{\frac{p(1-p)}{N}}$$

typická odchylka  $f$  od  $p$ ; 100× víc pokusů → 10× menší odchylka $t \propto N$  doba simulace roste přímo úměrně počtu opakování

$$P(\text{aspoň jednou}) = 1 - (1 - p)^n$$

jev s pravděpodobností  $p$  nastane aspoň jednou v  $n$  nezávislých pokusech

$$\pi \approx 4 \cdot \frac{\text{zásahy}}{N}$$

Monte Carlo: body ve čtverci, zásah = bod ve čtvrtkruhu

**Součet dvou kostek (36 stejně možných dvojic)**

| Součet   | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|----------|---|---|---|---|---|---|---|---|----|----|----|
| Možností | 1 | 2 | 3 | 4 | 5 | 6 | 5 | 4 | 3  | 2  | 1  |

**Poučky a postupy**

- `import random`; `hod = random.randint(1, 6)` – obě meze patří do výsledku (na rozdíl od `range`).
- Četnosti sbírej do seznamu počítadel: `cetnosti = [0] * 7` a `cetnosti[hod] += 1` (index 0 se nevyužije).
- Relativní četnost porovnej s teorií  $\frac{1}{6} \doteq 0,1667$ ; odchylku počítej `abs(f - 1 / 6)`.
- Malý počet pokusů dává „rozházené“ výsledky; přesnost roste jen s odmocninou z počtu pokusů.
- 10× víc opakování → asi 10× delší výpočet; při 100× víc opakování je odhad jen asi 10× přesnější.
- Stačí-li četnosti, neukládej všechny hody do seznamu – ušetříš paměť i čas.
- `random.seed(číslo)` použij při ladění a testech, aby šel výsledek zopakovat.

**Pozor na**

- `random.randrange(1, 6)` nikdy nedá 6 – pro kostku `randint(1, 6)`.
- Součet dvou kostek nemodeluj `randint(2, 12)` – součty nejsou stejně pravděpodobné (tabulka).
- Simulace nic nedokazuje, jen odhaduje; stejný program dává při každém spuštění jiné číslo.
- `print` uvnitř dlouhého cyklu výpočet mnohonásobně zpomalí – vypisuj až po cyklu.

**Vzorový příklad**

Simuluj 10 000 hodů kostkou a porovnej relativní četnost šestek s teorií.

- `import random`; `N = 10000`; `sestky = 0`.
- `for i in range(N): if random.randint(1, 6) == 6: sestky += 1`.
- `f = sestky / N` – např. 0.1683 (při každém spuštění jiné číslo).
- Teorie:  $p = \frac{1}{6} \doteq 0,1667$ ; odchylka  $|f - p| \doteq 0,002$ ,  
typicky do  $\sigma = \sqrt{\frac{p(1-p)}{N}} \doteq 0,004$ .
- Pro  $N = 100\,000$  bude odchylka asi  $\sqrt{10} \doteq 3\times$  menší, výpočet ale potrvá asi 10× déle.